THE LAWS OF SOFTWARE TEAMS

Conway's law

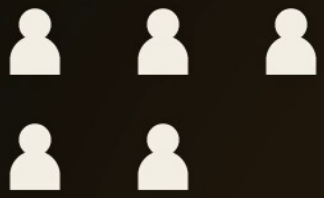
Organizations design systems that copy their communication structure

swipe →

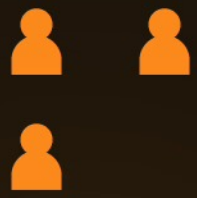
Ivan Rykov

1968 · 8 people, 2 compilers

Teams in
Same shape out



COBOL · 5 people



ALGOL · 3 people

phase 1	phase 1
phase 2	phase 2
phase 3	phase 3
phase 4	
phase 5	

Melvin Conway, “How Do Committees Invent?”, Datamation, 1968

Organizations design systems that copy their own communication structure

Conway’s law · paraphrased from the 1968 paper

architecture



Teams that don't talk ship code that doesn't fit



Glue code piles up in between

Microsoft · Windows Vista

The org chart predicted the bugs

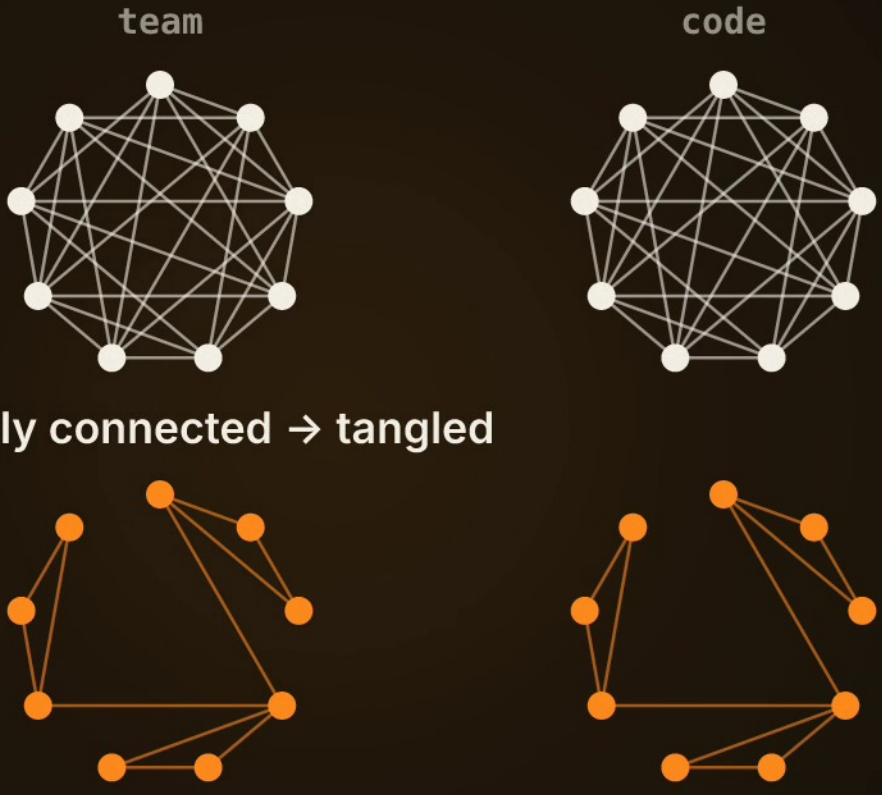


3,404 binaries · 50M+ lines of code

86% precision from org structure, better than churn, complexity, coverage

Nagappan, Murphy & Basili, ICSE 2008 · 1 dot ≈ 10 binaries

Your code is a mirror



MacCormack, Rusnak & Baldwin · HBS & MIT · diagram illustrative

Flip it: design the teams first



Amazon, ~2002: teams talk only through service interfaces

inverse Conway maneuver: LeRoy & Simons, 2010 · Amazon: Yegge, 2011

Flo: logic moved to where the team is



each team: 1 iOS · 1 Android · 2-4 backend

app store release — every few weeks

backend-driven releases

20-30 a day

via manager.dev

Want a different architecture?



Draw a different org chart

WHAT I TAKE FROM THIS

The inverse Conway maneuver applies the law on purpose: shape the teams for the system you want. This is how I do it:

1. **A team is built around a product or a domain: the problem it solves and the result it owns. It should go from an idea to a working change without waiting for other teams. In product work, cross-functional teams are usually better than teams split into frontend, backend and QA**
2. **If most tasks need several teams, or the overlap between teams keeps growing, I first find where work gets stuck. Then I review the boundaries**
3. **Every service or module has one owner: a team or a few named people, with the authority and the time to support it. Shared code needs an owner too. When ownership moves to another team, there is a clear date and a full handover. Open-ended "both teams own it" means nobody owns it**

WHAT I TAKE FROM THIS

4. **Each team can release and roll back its own changes. Where possible, an API contract between teams changes step by step: first the new version, then consumers move to it, then the old one is removed**
5. **When several teams repeat the same work, a platform team helps them do it on their own. It adds an approval step only for changes that can cause serious damage, for example to money, security or reliability. Otherwise approval just blocks the teams**
6. **Ownership comes first, services come later. By default, a small team works in a modular monolith with a clear owner for each part. Each owner can still change its part without waiting for the others. A new service adds work, so it needs a clear benefit**

WHAT I TAKE FROM THIS

7. **Dependent teams talk directly about contracts and shared decisions. Decisions and their reasons are written where everyone can see them. I cut meetings where people just report status, and approvals that are a formality**

AI agents let a small team handle more code, but code is only part of the load. People still have to understand the product, check decisions and run the system. So I widen a team's ownership only when it can handle review and support, not because it writes code faster. If a decision needs constant agreement between teams, faster code does not remove that dependency. Faster coding may only make the dependency more visible

Sources: Melvin Conway, Datamation, 1968 · LeRoy and Simons, 2010 · Nagappan, Murphy and Basili, 2008 · MacCormack, Rusnak and Baldwin · Yegge on Amazon · manager.dev on Flo

next: Dunbar's number